Pimpri Chinchwad Education Trust's
Pimpri Chinchwad College of Engineering
(An Autonomous Institute Affiliated to SPPU, Pune)

DevOps - Kubernetes & Helm Container Orchestration Laboratory Assignment 9

Student Name: Rutuja Jagtap | PRN: 123B1B146 | Batch: C1
Department: Computer Engineering | Academic Year: 2026-2027 | Semester: II

Explore Container Orchestration using Kubernetes, Architecture, and Helm Chart Management

Kubernetes Cluster Architecture, Control Plane, Worker Nodes, Helm Chart Anatomy (Chart.yaml, values.yaml, templates/), and Lifecycle Commands (install, upgrade, uninstall, template)

1. Aim

To explore the concept of container orchestration using Kubernetes, evaluate its control-plane and worker-node architecture, and examine the key components of a Helm chart (Chart.yaml, values.yaml, templates/) while performing Helm operations including install, upgrade, uninstall, and template.

2. Problem Statement

Deploying and managing microservice topologies on bare container hosts leads to severe configuration drift, manual scaling bottlenecks, lack of self-healing, and unmanageable YAML manifest sprawl. The task is to analyze container orchestration mechanisms in Kubernetes, evaluate its distributed control-plane and worker-node architecture, structure application manifests using a modular Helm chart (Chart.yaml, values.yaml, templates/), and execute full lifecycle release management through Helm commands: install, upgrade, template, and uninstall.

3. Learning Objectives

Area	Learning Objective
Container Orchestration Principles	Understand automated bin-packing, self-healing, horizontal scaling, service discovery, and rolling zero-downtime updates.
Kubernetes Architecture	Study distributed master-worker topologies, control-plane reconciliation loops, and worker-node container execution.
Core System Components	Identify responsibilities of API Server, etcd, Scheduler, Controller Manager, kubelet, kube-proxy, and CRI.
Helm Package Management	Understand Helm v3 client architecture, package versioning, values injection, and release state management.
Helm Chart Anatomy	Examine and author core chart files: Chart.yaml (metadata), values.yaml (parameters), and templates/ (manifest blueprints).
Lifecycle CLI Operations	Execute and verify helm template (dry-run), helm install (release), helm upgrade (scaling/config), and helm uninstall (teardown).

4. Container Orchestration and Kubernetes Overview

Containerization encapsulates application runtimes and dependencies into immutable images. However, when deploying microservices across distributed server clusters, container engines alone are insufficient. Production workloads require container orchestration: automated scheduling, automated recovery upon node failures, dynamic load balancing, zerodowntime rolling deployments, and centralized secret and configuration distribution.

Kubernetes (K8s) provides this orchestration substrate through a declarative desired-state model. Operators declare the desired cluster state using declarative manifests, and Kubernetes continuously executes reconciliation control loops to ensure actual running infrastructure matches the target specification. Helm enhances Kubernetes by providing a package manager that parameterizes, versions, and bundles multi-manifest topologies into reusable packages called Helm Charts.

5. Kubernetes Architecture, Control Plane, and Helm Concepts

A Kubernetes cluster is composed of a Control Plane (responsible for global cluster decisions, scheduling, and event detection) and one or more Worker Nodes (responsible for maintaining running Pod workloads).

5.1 Kubernetes Multi-Node Cluster Architecture

Figure 1 illustrates the distributed cluster architecture: Helm communicates over authenticated HTTPS with the kubeapiserver, which records state into etcd and instructs worker node kubelets to instantiate pods across worker nodes.

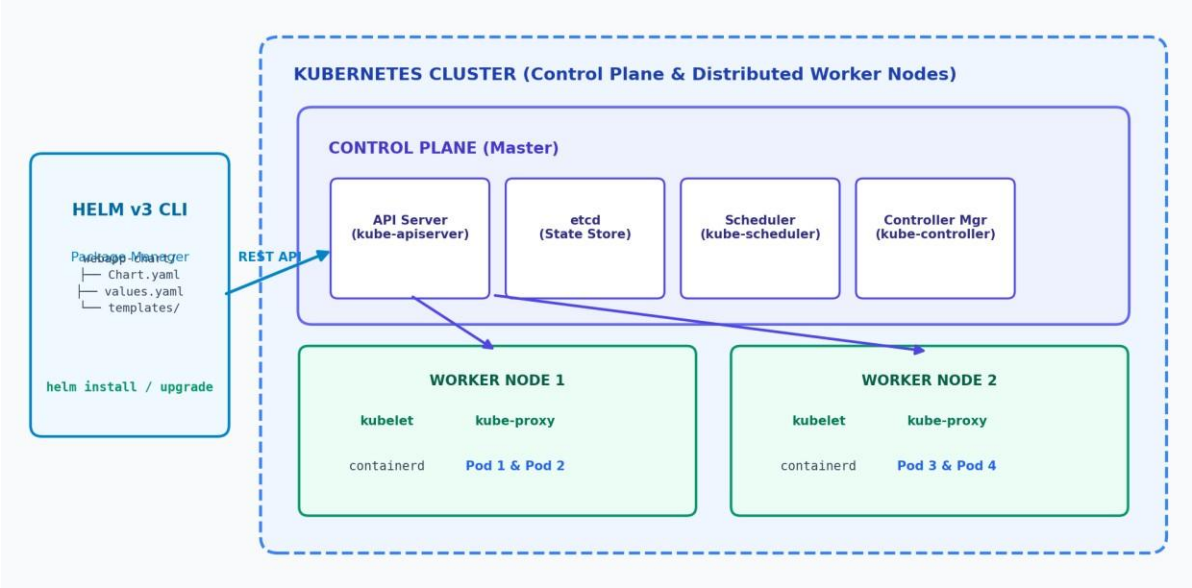


Figure 1. Distributed Kubernetes cluster architecture and Helm client integration topology

5.2 Kubernetes Control-Plane and Worker-Node Components

Component	Plane / Node	Primary Architectural Responsibility
API Server (kube-apiserver)	Control Plane	Exposes the Kubernetes REST API; authenticates, validates, and acts as the cluster communication hub.
etcd	Control Plane	Consistent, highly-available distributed key-value store holding all cluster state, secrets, and configurations.
Scheduler (kube-scheduler)	Control Plane	Evaluates unscheduled Pods and assigns them to optimal worker nodes based on resource capacity and affinity.
Controller Manager	Control Plane	Runs core background daemon controllers (Node, ReplicaSet, EndpointSlice) to enforce desired state.
kubelet	Worker Node	Primary node agent ensuring Pod containers described in PodSpecs are running, healthy, and reporting status.

kube-proxy	Worker Node	Maintains local network rules and IP tables, enabling Service IP communication and load-balanced pod traffic.
Container Runtime (containerd)	Worker Node	Implements the Container Runtime Interface (CRI) to pull images, create cgroups, and run containers.
Pod	Workload Unit	The smallest deployable computing unit in Kubernetes, hosting one or more closely coupled containers.

5.3 Helm Chart Anatomy and Packaging Architecture

Helm structures applications into standardized chart directories. The core components include:

- Chart.yaml: Contains chart metadata including API version (v2), chart name, semantic version, and application version.
- values.yaml: Defines default configuration values injected into the templates; easily overridden per deployment.
- templates/: Contains Go-template Kubernetes YAML manifests (Deployments, Services, ConfigMaps, Ingresses).
- _helpers.tpl: Template partials and helper macros that generate standardized labels and resource names.

5.4 Helm Templating and Release Lifecycle Pipeline

The Helm engine merges values.yaml with the templates/ directory using Go's text/template parser, producing standard Kubernetes manifests submitted to the API server:

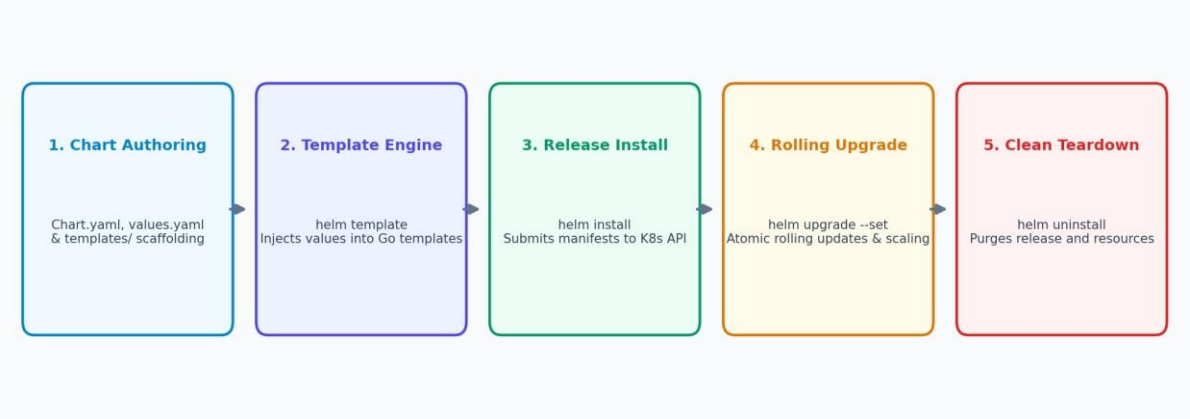


Figure 2. Helm chart template compilation and release lifecycle pipeline

6. Installation, Prerequisites, and Environment Configuration

The practical requires an active Kubernetes cluster (Minikube or multi-node cluster), kubectl CLI, and Helm v3.

6.1 Prerequisites Matrix

Component	Minimum Required Version	Verification Purpose
Kubernetes Multi-Node Cluster	v1.28.0+	Provides Control Plane and Worker Node environment.
kubectl CLI	v1.28.0+	Administrative command-line tool for inspecting Kubernetes cluster resources.
Helm v3 CLI	v3.12.0+	Package manager client responsible for chart templating and release management.
Container Runtime (containerd)	v1.7.0+	CRI runtime managing Pod execution on worker nodes.

6.2 Cluster & Tool Verification Commands

Verification Commands

```
helm version --short kubectl
version --client -o yaml kubectl
get nodes -o wide
```

6.3 Chart Directory Structure (webapp-chart)

Executing helm create webapp-chart produces the following project layout:

File / Directory	Role in Chart Package	Description
Chart.yaml	Chart Metadata	Declares chart API version (v2), package name, version, and appVersion.
values.yaml	Configurable Parameters	Specifies configurable values (replicaCount, image, service type, ports).
charts/	Chart Dependencies	Directory for storing dependent sub-charts.
File / Directory	**Role in Chart Package**	**Description**
templates/deployment.yaml	Deployment Manifest	Go template defining ReplicaSet, Pod template, and container ports.
templates/service.yaml	Service Manifest	Go template exposing Pods internally or via NodePort/ClusterIP.
templates/_helpers.tpl	Template Helpers	Shared Go template partials and reusable naming macros.

7. Helm Chart Anatomy: Metadata and Values Configuration

The chart behavior is parameterized through Chart.yaml and values.yaml:

7.1 Chart Metadata (webapp-chart/Chart.yaml) **webapp-chart/Chart.yaml**

```
apiVersion: v2 name:
webapp-chart
description: Cloud-native Nginx web application chart for DevOps Lab
type: application version: 0.1.0 appVersion: "1.16.0" maintainers:
  - name: Niraj Patil
email: 124B2B026@pccoe.edu
```

7.2 Default Values Configuration (webapp-chart/values.yaml) webapp-chart/values.yaml

```
replicaCount: 2
 image:   repository:
nginx   pullPolicy:
IfNotPresent   tag:
"1.25.4"
 service:
type: NodePort
port: 80
targetPort: 80
 resources:
limits:
    cpu: 250m
memory: 256Mi
requests:
cpu: 100m
memory: 128Mi
```

8. Kubernetes Template Manifests Implementation

The templates/ directory contains Go-template manifests dynamically evaluated by Helm:

8.1 Deployment Template (templates/deployment.yaml)

webapp-chart/templates/deployment.yaml

```
apiVersion: apps/v1 kind:
Deployment metadata:
  name: {{ .Release.Name }}-{{ .Chart.Name }}
labels:     app.kubernetes.io/instance: {{
.Release.Name }}
```

```
    app.kubernetes.io/name: {{ .Chart.Name }} spec:
replicas: {{ .Values.replicaCount }}   selector:
matchLabels:       app.kubernetes.io/instance: {{
.Release.Name }}   template:     metadata:
labels:         app.kubernetes.io/instance: {{
.Release.Name }}     spec:       containers:
- name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag
}}"           imagePullPolicy: {{ .Values.image.pullPolicy }}
ports:
- name: http
              containerPort: {{ .Values.service.targetPort }}
```

8.2 Service Template (templates/service.yaml)

webapp-chart/templates/service.yaml

```
apiVersion: v1 kind:
Service metadata:
  name: {{ .Release.Name }}-{{ .Chart.Name }}
labels:     app.kubernetes.io/instance: {{
.Release.Name }} spec:   type: {{
.Values.service.type }}   ports:
    - port: {{ .Values.service.port }}
targetPort: http       protocol: TCP
selector:
    app.kubernetes.io/instance: {{ .Release.Name }}
```

9. Helm Templating, Dry-Run, and Values Overrides

Executing helm template renders and previews the raw Kubernetes manifests without connecting to the cluster:

Dry-Run Templating & Linting

```
helm template webapp-release ./webapp-chart --set replicaCount=3 helm
lint ./webapp-chart
```

The helm lint command asserts chart syntax integrity and best practices, confirming 1 chart(s) linted, 0 chart(s) failed.

10. Execution Workflow

The comprehensive sequence of Helm and kubectl commands executed during Assignment 9:

Step #	Command Executed	Operational Purpose	Verification Point
1	helm version --short	Verifies Helm CLI build version.	Returns v3.15.4.
2	kubectl cluster-info	Verifies Control Plane API server endpoint connectivity.	Returns running Control Plane and CoreDNS URLs.
3	helm create webapp-chart	Scaffolds standard Helm chart directory structure.	Generates Chart.yaml, values.yaml, and templates/.
4	helm lint ./webapp-chart	Lints chart syntax and verifies structural conventions.	Confirms 0 chart(s) failed.
Step #	**Command Executed**	**Operational Purpose**	**Verification Point**
5	helm template webapp-release ./webapp-chart	Locally renders templates without cluster deployment.	Outputs resolved Kubernetes Deployment and Service YAML.
6	helm install webapp-release ./webapp-chart	Deploys chart as release 'webapprelease' on Kubernetes.	Release status shows deployed (Revision 1).
7	helm list -o table	Lists all active Helm releases in the default namespace.	Displays release webapp-release, chart version, and appVersion.
8	kubectl get pods,deployments,svc	Inspects live Pods, Deployments, and Services in cluster.	Confirms 2 Pods Running and NodePort service active.
9	helm upgrade webapp-release --set replicaCount=4	Performs rolling upgrade scaling replicas to 4.	Release revision increments to 2; 3rd & 4th pods start.
10	helm status webapp-release	Queries status, revision history, and notes of the release.	Displays Revision 2 status and deployment manifest.

11	helm uninstall webapp-release	Uninstalls release and removes all associated resources.	Pods and Service terminated cleanly; release purged.

11. Security, Best Practices and Good Practices

Practice	Kubernetes & Helm Implementation & Application
Values Separation	Never store environment-specific or secret parameters in templates; separate into values.yaml or external Vault.
Linting & Dry-Runs	Execute helm lint and helm template before running helm install in production pipelines to catch schema errors.
Resource Quotas	Always declare resources.requests and limits in templates to prevent rogue pods starving worker node resources.
Deterministic Versioning	Adhere to Semantic Versioning (SemVer 2.0.0) in Chart.yaml (version for chart package, appVersion for container).
Namespace Isolation	Deploy applications into dedicated Kubernetes namespaces (-n mynamespace) rather than the default namespace.
Atomic Operations	Utilize the --atomic flag during install and upgrade so failed deployments automatically roll back to previous healthy revisions.

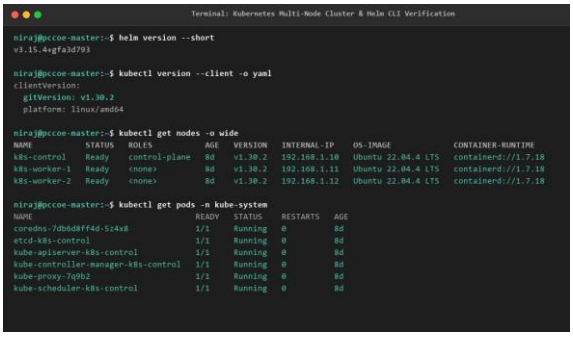
12. Practical Verificatio ı Checklist

Activity	Verification Point	Evidence to Attach
Cluster & Tool Setup	Kubernetes nodes Ready; Helm v3 installed and verified.	CLI version and cluster node screenshot (Evidence A).
Chart Scaffolding	webapp-chart directory created; helm lint and template succeed.	Chart linting and dry-run template screenshot (Evidence B).
Release Installation	helm install succeeds; 2 Pods running and Service active.	Helm release list and kubectl get pods screenshot (Evidence C).
Upgrade & Teardown	helm upgrade scales replicas to 4; helm uninstall cleans up.	Upgrade scaling and uninstall confirmation screenshot (Evidence D).

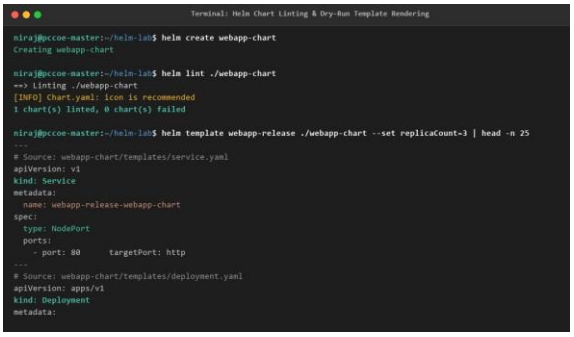
13. Practical Evidence

The terminal captures below record the complete practical execution of container orchestration using Kubernetes and Helm chart management, verifying cluster health, chart templating, release deployment, rolling upgrade scaling to 4 replicas, and clean teardown:

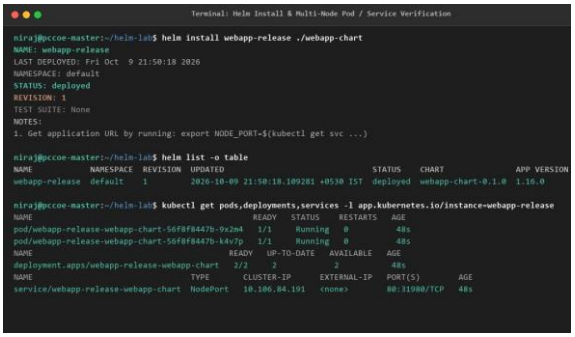
A. Multi-Node Cluster Health & Helm CLI Verification



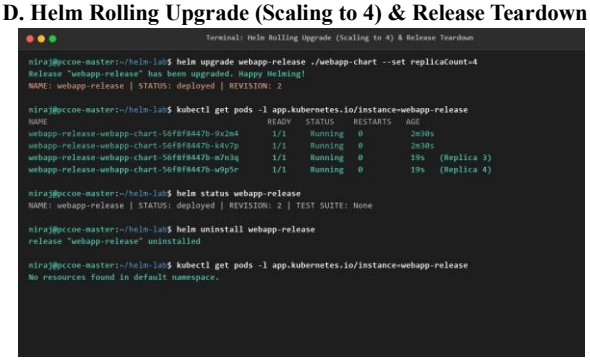
B. Helm Chart Scaffolding, Linting & Dry-Run Templating



C. Helm Chart Installation & Pod / Service Verification



D. Helm Rolling Upgrade (Scaling to 4) & Release Teardown



14. Result

The laboratory assignment successfully explored container orchestration concepts using Kubernetes and Helm chart management. The roles of the Control Plane (API Server, etcd, Scheduler, Controller Manager) and Worker Nodes (kubelet, kube-proxy, Container Runtime) were comprehensively evaluated. A custom Helm chart (webapp-chart) was scaffolded with Chart.yaml, values.yaml, and templates, validated via helm lint and helm template, deployed into the cluster as release 'webapp-release', scaled from 2 to 4 replicas via helm upgrade, and cleanly uninstalled.

15. Conclusion

Kubernetes container orchestration delivers enterprise-grade self-healing, automated bin-packing, and dynamic scaling for microservices, while Helm eliminates configuration drift by parameterizing manifests into reusable, version-controlled packages. By mastering Chart.yaml, values.yaml, and templates along with install, upgrade, template, and uninstall commands, developers and DevOps engineers can deploy predictable, scalable, and auditable cloud-native applications across multi-environment deployment lifecycles.